

Rotations: Euler Angles, Gimbal Lock, and Quaternions

A Comprehensive Guide for GNC Engineers

April 2026

Index

1	Euler Angle Rotations	3
1.1	Rotation Convention: Right-Hand Rule	3
1.2	Elementary Rotation Matrices	3
1.3	Intrinsic Rotations	4
1.4	Extrinsic Rotations	4
1.5	Key Equivalence	5
2	Nav-to-Body or Body-to-Nav?	5
2.1	Expanded Nav-to-Body Matrix (ZYX)	5
2.2	Expanded Body-to-Nav Matrix (ZYX)	6
2.3	Passive vs. Active — Two Conventions, Same Product Order	6
2.4	Active vs. Passive — Worked Example	9
2.5	Intrinsic/Extrinsic vs. Active/Passive — Two Independent Choices	11
2.6	Intrinsic/Extrinsic for GNC Aerospace vs. CTCT	13
2.7	Active/Passive for GNC Aerospace vs. CTCT	13
3	Gimbal Lock	14
3.1	Definition	14
3.2	When Does It Happen?	14
3.3	Axis Alignment at the Singularity	14
3.4	Proof: Why Exactly One DOF Is Lost	15
3.5	Analogy: Latitude and Longitude at the Pole	16
3.6	Consequences	16
3.7	What “Losing a DOF” Does and Does Not Mean	16
3.8	Detection	17
3.9	Mitigation Strategies	17
4	Intrinsic vs Extrinsic: Axis Alignment Comparison	17
5	MATLAB Example	17

6	Quaternions	18
6.1	Definition	18
6.2	Multiplication Rules	18
6.3	Quaternion Arithmetic	19
6.4	Unit Quaternions as Rotations	19
6.5	Composing Rotations	19
6.6	Conversions	20
6.7	Double Cover of $SO(3)$	20
6.8	Quaternion Derivative — Rotational Kinematics	20
6.9	SLERP — Spherical Linear Interpolation	21
6.10	Quaternion Error and Attitude Control	21
6.11	Exponential and Logarithmic Maps	21
6.12	Euler Angles vs. Quaternions	21
6.13	Practical Tips for GNC	21
6.14	MATLAB Examples	22
7	Denavit–Hartenberg (DH) Parameters	23
7.1	The Four DH Parameters	23
7.2	The DH Transformation Matrix	23
7.3	Rotary (Revolute) vs. Linear (Prismatic) Joints	24
7.4	Forward Kinematics — End Effector Position	25
7.5	Summary of DH Procedure	26

1 Euler Angle Rotations

Any orientation in 3D space can be described by a sequence of three successive rotations about coordinate axes. These are called **Euler angles**. The two fundamental conventions for defining these rotations are *intrinsic* and *extrinsic*.

1.1 Rotation Convention: Right-Hand Rule

This document uses the **right-hand rule** throughout, which is the standard convention in aerospace, GNC, and most engineering disciplines.

Right-Hand Rule

Point your **right thumb** along the positive direction of the rotation axis. Your fingers curl in the direction of **positive rotation** (counter-clockwise when viewed from the tip of the axis).

	Right-Hand Rule	Left-Hand Rule
Positive rotation	Counter-clockwise (looking down axis)	Clockwise (looking down axis)
Cross product	$\hat{x} \times \hat{y} = +\hat{z}$	$\hat{x} \times \hat{y} = -\hat{z}$
Used in	Aerospace, physics, most GNC	Some computer graphics (e.g. DirectX)
Matrix form	$\sin \theta$ terms as shown below	$\sin \theta$ terms negated

Table 1: Right-hand vs. left-hand rotation conventions.

Convention matters

Mixing right-hand and left-hand conventions silently negates rotation directions. Always verify which convention your codebase, sensor, and simulation tool use before combining rotations.

1.2 Elementary Rotation Matrices

Using the right-hand rule, the three elementary (principal-axis) rotation matrices are:

Rotation about the X-axis by angle ϕ (roll)

$$R_X(\phi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & \sin \phi \\ 0 & -\sin \phi & \cos \phi \end{pmatrix} \quad (1)$$

Rotation about the Y-axis by angle θ (pitch)

$$R_Y(\theta) = \begin{pmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{pmatrix} \quad (2)$$

Rotation about the Z-axis by angle ψ (yaw)

$$R_Z(\psi) = \begin{pmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (3)$$

Sign pattern

In the right-hand convention for nav-to-body matrices, the sin term with the **negative sign** appears:

- R_X : row 3, column 2 $(-\sin \phi)$
- R_Y : row 1, column 3 $(-\sin \theta)$
- R_Z : row 2, column 1 $(-\sin \psi)$

For a left-hand convention, these signs flip.

1.3 Intrinsic Rotations

Intrinsic rotations are performed about the **axes of the rotating body** (body-fixed frame). Each successive rotation occurs about an axis that has been updated by the previous rotation.

For the **ZYX intrinsic sequence** (common in aerospace as yaw–pitch–roll):

$$R = R_Z(\psi) R_Y(\theta) R_X(\phi) \quad (4)$$

1. Rotate about the body's **Z** axis by ψ (yaw).
2. Rotate about the **new** Y' axis by θ (pitch).
3. Rotate about the **new** X'' axis by ϕ (roll).

1.4 Extrinsic Rotations

Extrinsic rotations are performed about the **axes of the fixed/world frame**. The axes never change between rotations.

For the **XYZ extrinsic sequence**:

$$R = R_Z(\gamma) R_Y(\beta) R_X(\alpha) \quad (5)$$

1. Rotate about the fixed **X** axis by α .
2. Rotate about the fixed **Y** axis by β .
3. Rotate about the fixed **Z** axis by γ .

1.5 Key Equivalence

Intrinsic–Extrinsic Equivalence

An **intrinsic** rotation sequence in order A - B - C produces the **same** result as an **extrinsic** rotation in the **reversed** order C - B - A with the same angles:

$$R_{\text{intrinsic}}(Z\text{-}Y\text{-}X) = R_{\text{extrinsic}}(X\text{-}Y\text{-}Z) \quad (6)$$

This holds because:

- **Intrinsic**: multiply rotation matrices left-to-right in the order applied $\Rightarrow R_Z \cdot R_Y \cdot R_X$.
- **Extrinsic**: multiply rotation matrices right-to-left in the order applied $\Rightarrow$ same product $R_Z \cdot R_Y \cdot R_X$.

	Intrinsic	Extrinsic
Axes	Body-fixed (rotate with body)	World-fixed (stationary)
Matrix order	Multiply in sequence order	Multiply in reverse order
Common use	Aerospace (yaw–pitch–roll)	Animation, some robotics
Gimbal lock	Middle rotation = ± 90	Same condition, same angles

Table 2: Comparison of intrinsic and extrinsic conventions.

2 Nav-to-Body or Body-to-Nav?

With the standard intrinsic ZYX convention the composed matrix

$$R_{\text{intrinsic}} = R_Z(\psi) R_Y(\theta) R_X(\phi) \quad (7)$$

is the **nav-to-body** (world-to-body) rotation matrix. It transforms a vector from the navigation/world frame into the body frame:

$$\vec{v}_{\text{body}} = R_{\text{intrinsic}} \vec{v}_{\text{nav}} \quad (8)$$

To go **body-to-nav**, use the transpose (rotation matrices are orthogonal, so $R^{-1} = R^T$):

$$R_{\text{body} \rightarrow \text{nav}} = R_{\text{intrinsic}}^T = R_X(-\phi) R_Y(-\theta) R_Z(-\psi) \quad (9)$$

2.1 Expanded Nav-to-Body Matrix (ZYX)

Multiplying $R_Z(\psi) R_Y(\theta) R_X(\phi)$ using the elementary matrices from Equations (3), (2), (1):

$$R_{\text{nav} \rightarrow \text{body}} = \begin{pmatrix} \cos \theta \cos \psi & \cos \theta \sin \psi & -\sin \theta \\ \sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi & \sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi & \sin \phi \cos \theta \\ \cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi & \cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi & \cos \phi \cos \theta \end{pmatrix} \quad (10)$$

where ψ = yaw, θ = pitch, ϕ = roll (all right-hand positive).

2.2 Expanded Body-to-Nav Matrix (ZYX)

Since R is orthogonal, $R_{\text{body} \rightarrow \text{nav}} = R_{\text{nav} \rightarrow \text{body}}^T$:

$$R_{\text{body} \rightarrow \text{nav}} = \begin{pmatrix} \cos \theta \cos \psi & \sin \phi \sin \theta \cos \psi - \cos \phi \sin \psi & \cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi \\ \cos \theta \sin \psi & \sin \phi \sin \theta \sin \psi + \cos \phi \cos \psi & \cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi \\ -\sin \theta & \sin \phi \cos \theta & \cos \phi \cos \theta \end{pmatrix} \quad (11)$$

Quick check

At zero attitude ($\psi = \theta = \phi = 0$), both matrices reduce to the 3×3 identity, confirming body frame = nav frame.

Why nav-to-body?

The intrinsic ZYX sequence describes how to rotate *from* the nav frame *into* the body frame: first yaw about nav Z, then pitch about the intermediate Y' , then roll about the final body X'' . The composed matrix maps nav-frame coordinates into the resulting body frame.

2.3 Passive vs. Active — Two Conventions, Same Product Order

The product $R_Z(\psi) R_Y(\theta) R_X(\phi)$ can yield *either* nav-to-body or body-to-nav, depending on how the elementary matrices are defined. Both use the right-hand rule and the **same multiplication order**; the only difference is the **sign of sin** in each elementary matrix.

GNC Aerospace Convention — Passive (Alias)

Standard aerospace textbooks (Titterton & Weston, Stevens & Lewis, etc.) use the **passive** (alias) elementary matrices listed in Equations (3)–(1):

$$R_Z^{(P)}(\psi) = \begin{pmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (+\sin \text{ above diagonal}) \quad (12)$$

The composed product gives the **nav-to-body** direction-cosine matrix:

$$R_Z^{(P)} R_Y^{(P)} R_X^{(P)} = C_n^b \quad (\text{nav-to-body}) \quad (13)$$

Meaning: the matrix *re-expresses* a stationary vector in the rotated body frame. The frame moves; the vector stays.

$$\vec{v}_{\text{body}} = C_n^b \vec{v}_{\text{nav}}$$

CTCT Convention — Active (Alibi)

The CTCT codebase (e.g. `Rotation_Matrix.m`, `DozerMinMaxYawCalTestHelpers.h`, autocode `Rot_Mtrx_YRP`) uses **active** (alibi) elementary matrices with the opposite sin sign.

CTCT Angle Naming

CTCT uses non-standard angle names relative to aerospace:

Axis	Aerospace name	CTCT name	Symbol
Z	Yaw	Yaw	ψ
Y	Pitch	Roll	ϕ_R
X	Roll	Pitch	θ_P

The intrinsic sequence is therefore **Yaw–Roll–Pitch** in CTCT terminology vs. **Yaw–Pitch–Roll** in aerospace. The matrix product $R_Z R_Y R_X$ is identical—only the labels differ.

The three CTCT active elementary matrices are:

CTCT: Rotation about Z by ψ (Yaw)

$$R_Z^{(A)}(\psi) = \begin{pmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (+\sin \text{ below diagonal}) \quad (14)$$

CTCT: Rotation about Y by ϕ_R (Roll)

$$R_Y^{(A)}(\phi_R) = \begin{pmatrix} \cos \phi_R & 0 & \sin \phi_R \\ 0 & 1 & 0 \\ -\sin \phi_R & 0 & \cos \phi_R \end{pmatrix} \quad (15)$$

CTCT: Rotation about X by θ_P (Pitch)

$$R_X^{(A)}(\theta_P) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \theta_P & -\sin \theta_P \\ 0 & \sin \theta_P & \cos \theta_P \end{pmatrix} \quad (16)$$

Active sign pattern

Compare with the passive (aerospace) elementary matrices in Equations (1)–(3): each active matrix is the **transpose** of its passive counterpart. The $-\sin$ terms move to the opposite off-diagonal position:

- $R_X^{(A)}$: $-\sin \theta_P$ at row 2, col 3 (passive: row 3, col 2)
- $R_Y^{(A)}$: $-\sin \phi_R$ at row 3, col 1 (passive: row 1, col 3)
- $R_Z^{(A)}$: $-\sin \psi$ at row 1, col 2 (passive: row 2, col 1)

Using the **same multiplication order** $R_Z R_Y R_X$ gives the **body-to-nav** matrix:

$$\boxed{R_Z^{(A)}(\psi) R_Y^{(A)}(\phi_R) R_X^{(A)}(\theta_P) = C_b^m \quad (\text{body-to-nav})} \quad (17)$$

Meaning: the matrix physically *moves* a body-frame vector into the nav frame. The vector moves; the frame stays.

$$\vec{v}_{\text{nav}} = C_b^m \vec{v}_{\text{body}}$$

Expanded CTCT Body-to-Nav Matrix

Multiplying $R_Z^{(A)}(\psi) R_Y^{(A)}(\phi_R) R_X^{(A)}(\theta_P)$:

$$C_b^m = \begin{pmatrix} \cos \phi_R \cos \psi & -\cos \theta_P \sin \psi + \sin \theta_P \sin \phi_R \cos \psi & \sin \theta_P \sin \psi + \cos \theta_P \sin \phi_R \cos \psi \\ \cos \phi_R \sin \psi & \cos \theta_P \cos \psi + \sin \theta_P \sin \phi_R \sin \psi & -\sin \theta_P \cos \psi + \cos \theta_P \sin \phi_R \sin \psi \\ -\sin \phi_R & \sin \theta_P \cos \phi_R & \cos \theta_P \cos \phi_R \end{pmatrix} \quad (18)$$

where ψ = Yaw, ϕ_R = Roll (CTCT, about Y), θ_P = Pitch (CTCT, about X). This is the **transpose** of the aerospace nav-to-body matrix (Eq. 10) with the CTCT naming applied.

Why the Same Order Gives Transposes

Each active elementary matrix is the transpose of its passive counterpart: $R_{\text{axis}}^{(A)} = (R_{\text{axis}}^{(P)})^T$.

A composed product of individually-transposed factors is **not** the same as transposing the whole product—the algebraic transpose rule $(ABC)^T = C^T B^T A^T$ reverses the order. So how can the same-order active product equal the transpose of the same-order passive product?

The answer is that the expanded 3×3 matrix happens to satisfy the identity:

$$\boxed{R_Z^{(A)} R_Y^{(A)} R_X^{(A)} = (R_Z^{(P)} R_Y^{(P)} R_X^{(P)})^T} \quad (19)$$

This can be verified element-by-element: flipping the sign of every sin term in every elementary matrix and then multiplying in the same order produces exactly the transpose of the passive product.

Numerical verification

At $\psi = 30^\circ$, $\theta = 20^\circ$, $\phi = 10^\circ$:
 $C_n^b = R_Z^{(P)} R_Y^{(P)} R_X^{(P)}$: $(1, 2) = +0.4699$, $(2, 1) = -0.4410$
 $C_b^n = R_Z^{(A)} R_Y^{(A)} R_X^{(A)}$: $(1, 2) = -0.4410$, $(2, 1) = +0.4699$

These are transposes of each other. The reversed-order product $R_X^{(A)} R_Y^{(A)} R_Z^{(A)}$ gives $(1, 2) = -0.4699$, $(2, 1) = +0.5438$ —a **completely different** matrix.

Reconciling with the Transpose Reversal Rule

The algebraic identity $(ABC)^T = C^T B^T A^T$ is always true. So:

$$(R_Z^{(P)} R_Y^{(P)} R_X^{(P)})^T = R_X^{(A)} R_Y^{(A)} R_Z^{(A)} \quad (\text{reversed order, purely algebraic}) \quad (20)$$

This reversed product $R_X^{(A)} R_Y^{(A)} R_Z^{(A)}$ is an X - Y - Z rotation sequence—a *different* set of physical rotations than Z - Y - X . Meanwhile, the same-order product $R_Z^{(A)} R_Y^{(A)} R_X^{(A)}$ is a Z - Y - X sequence with individually-transposed factors. Both happen to produce the *same* numerical matrix (which is C_b^n), but through different mathematical paths. There is no contradiction.

Side-by-Side Comparison

	GNC Aerospace (Passive)	CTCT (Active)
Elementary R_Z	$+\sin \psi$ above diagonal	$+\sin \psi$ below diagonal
Product order	$R_Z R_Y R_X$	$R_Z R_Y R_X$ (same)
Result of product	C_b^n (nav-to-body)	C_b^n (body-to-nav)
To get the other direction	Transpose: $C_b^n = (C_n^b)^T$	Transpose: $C_n^b = (C_b^n)^T$
Key relationship	$R_Z^{(A)} R_Y^{(A)} R_X^{(A)} = (R_Z^{(P)} R_Y^{(P)} R_X^{(P)})^T$	

Table 3: GNC Aerospace vs. CTCT conventions—same product order, transposed elementary matrices, transposed result.

Common pitfall

Both conventions are right-hand positive and use the same matrix product order $R_Z R_Y R_X$. The *only* difference is the sign of $\sin$ in the elementary matrices. Mixing conventions silently transposes the result, which can appear as a sign error in yaw or roll and is notoriously hard to debug.

Quick Diagnostic

Compute $R_Z(90^\circ) \hat{x}$ where $\hat{x} = [1, 0, 0]^T$:

- **GNC Aerospace** (passive): result = $[0, -1, 0]^T \Rightarrow$ the nav $\hat{x}$ has body coordinate $-\hat{y}$ (nav-to-body).
- **CTCT** (active): result = $[0, +1, 0]^T \Rightarrow$ the body $\hat{x}$ points along nav $+\hat{y}$ (body-to-nav).

This document's convention

All matrices in this document use the **GNC Aerospace** (passive) convention. Therefore $R_Z(\psi) R_Y(\theta) R_X(\phi) = C_b^n$. The body-to-nav matrix is obtained via transpose: $C_n^b = (C_b^n)^T$.

2.4 Active vs. Passive — Worked Example

To make the distinction concrete, consider a single 90° yaw rotation applied to a vector $\vec{v} = [1, 0, 0]^T$ (pointing along the X -axis).

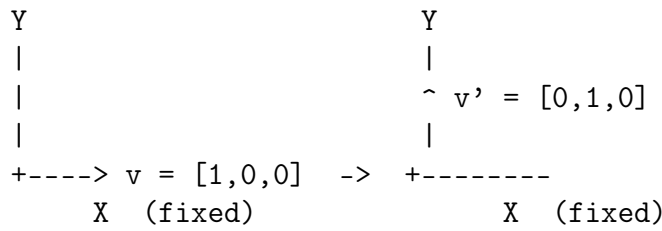
Active (CTCT) — “Move the vector”

The vector physically rotates within a **fixed** frame. The coordinate axes do not move; the arrow does.

$$R_Z^{(\text{active})}(90^\circ) = \begin{pmatrix} \cos 90^\circ & -\sin 90^\circ & 0 \\ \sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (21)$$

$$\vec{v}' = R_Z^{(\text{active})}(90^\circ) \vec{v} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad (22)$$

The arrow that was pointing along $+X$ now points along $+Y$. The axes did not move.



BEFORE

AFTER (active / CTCT)

Think of it as: you are standing still, watching an arrow on the ground rotate in front of you.

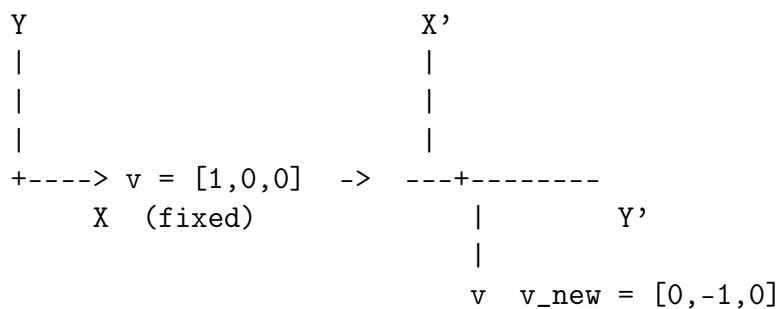
Passive (GNC Aerospace) — “Move the frame”

The vector stays **fixed in space**. The coordinate frame rotates underneath it, and we re-read the vector’s coordinates in the new frame.

$$R_Z^{(\text{passive})}(90^\circ) = \begin{pmatrix} \cos 90^\circ & \sin 90^\circ & 0 \\ -\sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (23)$$

$$\vec{v}_{\text{new}} = R_Z^{(\text{passive})}(90^\circ) \vec{v} = \begin{pmatrix} 0 \\ -1 \\ 0 \end{pmatrix} \quad (24)$$

The arrow is still pointing the same direction in space, but we rotated the coordinate frame 90° CCW. In this new frame the original arrow is at $[0, -1, 0]^T$ —it points along $-Y'$.



BEFORE
(old frame)

AFTER (passive / GNC Aerospace)
(new frame rotated 90 deg CCW)

Think of it as: the arrow is nailed to the floor. You rotate yourself 90° CCW. From your new perspective the arrow that was “forward” is now to your right—it reads $[0, -1, 0]^T$ in your rotated frame.

Key Relationship

The two matrices are **transposes** of each other for the same physical rotation:

$$\boxed{R^{(\text{passive})} = (R^{(\text{active})})^T} \quad (25)$$

Verify:

$$\begin{pmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}^T \quad \checkmark$$

3-D GNC Example

An aircraft has yaw $\psi = 90^\circ$ (nose pointing East). A GPS gives velocity in the nav frame: $\vec{v}_{\text{nav}} = [5, 0, 0]^T$ m/s (moving North).

- **CTCT (Active)** — “Where does the body $\hat{x}$ point in nav?”

$$R_Z^{(A)}(90^\circ) \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

Body $\hat{x}$ points along nav $+Y$ (East). This is C_b^n .

- **GNC Aerospace (Passive)** — “What is the nav velocity in body coords?”

$$R_Z^{(P)}(90^\circ) \begin{pmatrix} 5 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ -5 \\ 0 \end{pmatrix}$$

The 5 m/s North velocity appears as -5 m/s along body Y (coming from the right). This is C_n^b .

Summary

2.5 Intrinsic/Extrinsic vs. Active/Passive — Two Independent Choices

These two pairs of terms are often confused, but they answer **different, independent** questions:

	CTCT (Active/Alibi)	GNC Aerospace (Passive/Alias)
What moves?	The vector	The coordinate frame
What stays?	The frame	The vector
Result	$\vec{v}' = R^{(A)}\vec{v}$	$\vec{v}_{\text{new}} = R^{(P)}\vec{v}_{\text{old}}$
GNC use	C_b^n (body-to-nav)	C_n^b (nav-to-body)
Relationship	$R^{(P)} = (R^{(A)})^T$ — same physics, different bookkeeping	

Table 4: Active vs. passive rotation at a glance.

	What it decides	Options
Intrinsic vs. Extrinsic	Which axes to rotate about	Body-fixed (moving) vs. World-fixed (stationary)
Active vs. Passive	What the matrix does to a vector	Rotates the vector vs. Re-expresses it in a new frame

Table 5: The two independent convention choices.

Intrinsic vs. Extrinsic — “About which axes?”

This determines the **sequence of axes**:

- **Intrinsic (body-fixed):** Each rotation is about the *current* body axis, which moves with each step.
“Yaw about body Z , then pitch about the new Y' , then roll about the new X'' .”
- **Extrinsic (world-fixed):** Each rotation is about the *original* fixed world axis.
“Rotate about fixed X , then fixed Y , then fixed Z .”

Key fact: Intrinsic Z - Y - X = Extrinsic X - Y - Z (reversed axis order, same matrix product $R_Z R_Y R_X$). This choice affects **only the multiplication order**, nothing else.

Active vs. Passive — “What does the matrix do?”

This determines the **meaning of the result**:

- **Active (alibi):** The matrix physically *moves* the vector to a new position within the same coordinate frame.

$$\vec{v}' = R\vec{v} \quad \text{— the vector rotates, the frame stays.}$$

- **Passive (alias):** The matrix *re-expresses* the same (stationary) vector in a different coordinate frame.

$$\vec{v}_{\text{new frame}} = R\vec{v}_{\text{old frame}} \quad \text{— the frame rotates, the vector stays.}$$

This choice affects **only the sign of sin** in the elementary matrices (active and passive are transposes of each other).

2.6 Intrinsic/Extrinsic for GNC Aerospace vs. CTCT

Both GNC Aerospace and CTCT use the **same** axis sequence (intrinsic Z - Y - X) and the **same** matrix product $R_Z R_Y R_X$. The only naming difference is which physical quantity is assigned to the Y -axis and X -axis rotations.

	GNC Aerospace	CTCT
Axis sequence	Intrinsic Z - Y - X	Intrinsic Z - Y - X (same)
Extrinsic equivalent	Extrinsic X - Y - Z	Extrinsic X - Y - Z (same)
Product form	$R_Z R_Y R_X$	$R_Z R_Y R_X$ (same)
Angle-axis assignment (where they differ):		
Z -axis rotation	Yaw (ψ)	Yaw (ψ) (same)
Y -axis rotation	Pitch (θ)	Roll (ϕ_R) (swapped)
X -axis rotation	Roll (ϕ)	Pitch (θ_P) (swapped)
Intrinsic order	Yaw $\rightarrow$ Pitch $\rightarrow$ Roll	Yaw $\rightarrow$ Roll $\rightarrow$ Pitch
Extrinsic order	Roll $\rightarrow$ Pitch $\rightarrow$ Yaw	Pitch $\rightarrow$ Roll $\rightarrow$ Yaw

Table 6: Intrinsic/extrinsic comparison—same axis sequence Z - Y - X and same matrix product, but different angle names for the Y and X axes.

The intrinsic/extrinsic choice is **not** where the two conventions differ. They differ in (1) active vs. passive and (2) which physical angle (pitch or roll) is assigned to the Y and X axes.

2.7 Active/Passive for GNC Aerospace vs. CTCT

Here is where the conventions diverge:

	GNC Aerospace	CTCT
Interpretation	Passive (alias)	Active (alibi)
R_Z sign pattern	+ sin above diagonal (row 1, col 2)	+ sin below diagonal (row 2, col 1)
Product $R_Z R_Y R_X$ yields	C_n^b (nav-to-body)	C_b^n (body-to-nav)
What the matrix does	Re-expresses vector in body frame	Moves body vector into nav frame
To get the other direction	Transpose	Transpose

Table 7: Active/passive comparison—the **only** difference between GNC Aerospace and CTCT. Same angles, same order, transposed result.

Summary

GNC Aerospace and CTCT use the *same* rotation sequence (intrinsic Z-Y-X) and the *same* product order ($R_Z R_Y R_X$). They differ *only* in active vs. passive: each elementary matrix is the transpose of the other convention's, and the composed result is the transpose.

$$\underbrace{R_Z^{(A)} R_Y^{(A)} R_X^{(A)}}_{\text{CTCT: } C_n^b} = \underbrace{\left(R_Z^{(P)} R_Y^{(P)} R_X^{(P)} \right)^T}_{\text{Transpose of Aerospace: } (C_n^b)^T}$$

CTCT Pitch/Roll Name Swap

In CTCT code the Y-axis angle is called “Roll” and the X-axis angle is called “Pitch”—**swapped** relative to standard aerospace. This applies to `Rotation_Matrix.m`, `Calc_Rotation_NF_Platform1`, `EulerXYZBasis`, and most autocode. Newer C++ components (`DozerMinMaxYawCalibration`) use standard aerospace naming. Always check which naming convention a given file uses.

This document's choices

Intrinsic ZYX (body-fixed axes) + **Passive** (alias) $\implies R_Z R_Y R_X = C_n^b$ (nav-to-body).

3 Gimbal Lock

3.1 Definition

Gimbal lock occurs when two of the three rotation axes align, causing the system to **lose one degree of freedom**. The representation collapses from 3 independent rotations to effectively 2.

3.2 When Does It Happen?

For the intrinsic ZYX sequence, gimbal lock occurs when the **middle angle** (pitch) reaches

$$\theta = \pm 90^\circ \tag{26}$$

At $\theta = 90^\circ$ the Z and X axes become aligned, and yaw and roll become indistinguishable—only their *sum or difference* matters.

3.3 Axis Alignment at the Singularity

Starting with body frame = nav frame (Z up, X forward, Y right):

Case 1: $\theta = +90^\circ$ (Nose Straight Up)

After pitching by $+90^\circ$ about Y' :

$$R_Y(90^\circ) = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{pmatrix}$$

Body Axis	Points Along (Nav)	Physical Meaning
X'' (forward)	$+Z_{\text{nav}}$ (up)	Nose pointing straight up
Y'' (right)	$+Y_{\text{nav}}$	Unchanged
Z'' (belly)	$-X_{\text{nav}}$ (backward)	Belly faces backward

Gimbal Lock

The body X'' axis is now aligned with the nav Z axis:

$$X''_{\text{body}} \parallel Z_{\text{nav}}$$

Roll (about X'') and yaw (about Z_{nav}) now rotate about the **same axis**. One degree of freedom is lost.

Case 2: $\theta = -90^\circ$ (Nose Straight Down)

Body Axis	Points Along (Nav)	Physical Meaning
X'' (forward)	$-Z_{\text{nav}}$ (down)	Nose pointing straight down
Y'' (right)	$+Y_{\text{nav}}$	Unchanged
Z'' (belly)	$+X_{\text{nav}}$ (fwd)	Belly faces forward

Again $X'' \parallel Z_{\text{nav}}$ (anti-parallel). Now $(\psi + \phi)$ is the only observable combined parameter.

3.4 Proof: Why Exactly One DOF Is Lost

Expand the full rotation matrix at $\theta = 90^\circ$:

$$R = R_Z(\psi) R_Y(90^\circ) R_X(\phi) = \begin{pmatrix} 0 & \sin(\psi - \phi) & \cos(\psi - \phi) \\ 0 & \cos(\psi - \phi) & -\sin(\psi - \phi) \\ -1 & 0 & 0 \end{pmatrix} \quad (27)$$

Every entry depends only on $(\psi - \phi)$, never on ψ or ϕ individually. Therefore infinitely many (ψ, ϕ) pairs yield the same orientation:

You have 3 parameters (ψ, θ, ϕ) but only **2** of them independently change the orientation:

1. θ — but it is stuck at 90° .
2. $(\psi - \phi)$ — one combined number, not two.

That is **2 effective DOF**, not 3. One is lost.

ψ (yaw)	ϕ (roll)	$\psi - \phi$	Same matrix?
30°	0°	30°	✓
40°	10°	30°	✓
0°	−30°	30°	✓
100°	70°	30°	✓

Table 8: Multiple Euler angle triples mapping to the same orientation at $\theta = 90^\circ$.

3.5 Analogy: Latitude and Longitude at the Pole

Consider a map with latitude and longitude:

- At the **equator**, changing latitude moves you north/south and changing longitude moves you east/west. **Two independent directions.**
- At the **North Pole**, changing longitude does nothing—every longitude passes through the same point. Longitude is meaningless. **One independent direction only.**

The Euler angle singularity is the same idea. At $\theta = 90^\circ$, ψ and ϕ collapse into a single effective parameter ($\psi - \phi$), just as at the pole all longitudes collapse to one point.

3.6 Consequences

Effect	Description
Ambiguous decomposition	Infinitely many (ψ, ϕ) pairs produce the same rotation.
Numerical instability	<code>atan2</code> -based extraction becomes ill-conditioned near $\theta = \pm 90^\circ$.
Discontinuous outputs	Small physical rotations produce large jumps in extracted angles.
Not a physical limitation	The orientation itself is fine; only the <i>Euler angle representation</i> breaks.

Table 9: Effects of gimbal lock.

3.7 What “Losing a DOF” Does and Does Not Mean

It does **NOT** mean:

- × The body cannot physically reach certain orientations.
- × The body is stuck or constrained.
- × There are only 2 possible directions to move.

It **DOES** mean:

- ✓ The *representation* (three Euler angles) can no longer uniquely describe orientations near $\theta = 90^\circ$.

- ✓ Two of the three angle “knobs” (ψ and ϕ) turn the same physical axis.
- ✓ Small physical rotations about one axis produce wild swings or ambiguity in the extracted angles.

3.8 Detection

```

1 % After extracting pitch (theta) from a rotation matrix:
2 if abs(cos(theta)) < 1e-6
3     warning('Near gimbal lock--Euler angles unreliable');
4 end

```

Listing 1: Detecting gimbal lock in MATLAB.

3.9 Mitigation Strategies

Approach	Trade-off
Quaternions	No singularities, 4 parameters, must normalise. Standard in GNC/INS.
Rotation matrices (DCM)	No singularities, 9 parameters (6 redundant), must re-orthogonalise.
Different Euler order	Moves the singularity (e.g. ZYZ puts it at $\theta = 0^\circ$), does not eliminate it.
Clamp & flag	Keep Euler angles for display but clamp θ and flag the degenerate case.

Table 10: Strategies to avoid or mitigate gimbal lock.

Key Takeaway

Gimbal lock is an inherent limitation of **all** 3-parameter attitude representations (Euler angles, Rodrigues parameters). It cannot be “fixed”—only avoided by using a higher-dimensional representation (quaternions or DCMs) for computation, and reserving Euler angles for human-readable output where θ stays well away from $\pm 90^\circ$.

4 Intrinsic vs Extrinsic: Axis Alignment Comparison

5 MATLAB Example

```

1 % Intrinsic ZYX (yaw=30 deg, pitch=20 deg, roll=10 deg)
2 R_intrinsic = rotz(30) * roty(20) * rotx(10);
3
4 % Equivalent extrinsic XYZ (same angles, reversed order)
5 % The matrix product is identical:

```

	Intrinsic ZYX	Extrinsic XYZ
What aligns?	Body X'' aligns with nav Z	After fixed- Y rotation of 90° , fixed- Z rotation acts the same as the earlier fixed- X rotation
Locked axes	Roll (body X'') $\leftrightarrow$ Yaw (nav Z)	First rotation (fixed X) $\leftrightarrow$ Third rotation (fixed Z)
Lost DOF	Only $(\psi \pm \phi)$ observable	Only $(\alpha \pm \gamma)$ observable
Intuition	Easier: “two body axes collapse onto same nav axis”	Harder: “fixed-frame rotations become redundant”

Table 11: Gimbal lock comparison between intrinsic and extrinsic conventions.

```

6 R_extrinsic = rotz(30) * roty(20) * rotx(10);
7
8 % Verify
9 assert(norm(R_intrinsic - R_extrinsic) < 1e-12);
10
11 % Nav-to-body
12 v_body = R_intrinsic * v_nav;
13
14 % Body-to-nav (transpose)
15 v_nav = R_intrinsic' * v_body;

```

Listing 2: Intrinsic ZYX and equivalent extrinsic XYZ in MATLAB.

6 Quaternions

Quaternions provide a singularity-free, compact representation of 3D rotations. They use four parameters with one constraint ($\|q\| = 1$), avoiding the gimbal lock problem inherent in all three-parameter representations.

6.1 Definition

A quaternion is a four-dimensional hypercomplex number:

$$q = w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k} \quad (28)$$

where $w, x, y, z \in \mathbb{R}$ and the imaginary units satisfy:

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{i}\mathbf{j}\mathbf{k} = -1 \quad (29)$$

A quaternion is often written as a scalar–vector pair:

$$q = (w, \vec{v}), \quad \vec{v} = (x, y, z) \quad (30)$$

6.2 Multiplication Rules

From the imaginary-unit identities the basis multiplication table follows:

$\times$	i	j	k
i	-1	k	- j
j	- k	-1	i
k	j	- i	-1

Table 12: Hamilton quaternion basis multiplication table.

6.3 Quaternion Arithmetic

Addition (element-wise):

$$q_1 + q_2 = (w_1 + w_2, \vec{v}_1 + \vec{v}_2) \quad (31)$$

Multiplication (non-commutative):

$$q_1 q_2 = (w_1 w_2 - \vec{v}_1 \cdot \vec{v}_2, w_1 \vec{v}_2 + w_2 \vec{v}_1 + \vec{v}_1 \times \vec{v}_2) \quad (32)$$

The cross-product term $\vec{v}_1 \times \vec{v}_2$ makes quaternion multiplication **non-commutative**: $q_1 q_2 \neq q_2 q_1$ in general.

Conjugate:

$$q^* = (w, -\vec{v}) \quad (33)$$

Norm:

$$\|q\| = \sqrt{w^2 + x^2 + y^2 + z^2} \quad (34)$$

Inverse:

$$q^{-1} = \frac{q^*}{\|q\|^2} \quad (35)$$

For **unit quaternions** ($\|q\| = 1$), the inverse equals the conjugate: $q^{-1} = q^*$.

6.4 Unit Quaternions as Rotations

A unit quaternion encodes a rotation about axis $\hat{n}$ by angle θ :

$$q = \left(\cos \frac{\theta}{2}, \sin \frac{\theta}{2} \hat{n} \right) \quad (36)$$

The **half-angle** arises from the double-cover property (Section 6.7).

Rotating a Vector

To rotate a vector $\vec{p}$, embed it as a pure quaternion $p = (0, \vec{p})$, then compute:

$$p' = q p q^* \quad (37)$$

The vector part of p' is the rotated vector.

6.5 Composing Rotations

Applying rotation q_1 first, then q_2 , is a single quaternion multiplication:

$$q_{\text{total}} = q_2 q_1 \quad (38)$$

The product of two unit quaternions is again a unit quaternion, so no renormalisation is needed after composition (unlike numerical integration).

6.6 Conversions

Quaternion → Rotation Matrix

$$R = \begin{pmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{pmatrix} \quad (39)$$

Quaternion → Euler Angles (ZYX)

$$\phi = \text{atan2}(2(wx + yz), 1 - 2(x^2 + y^2)) \quad (40)$$

$$\theta = \arcsin(2(wy - zx)) \quad (41)$$

$$\psi = \text{atan2}(2(wz + xy), 1 - 2(y^2 + z^2)) \quad (42)$$

Axis-Angle → Quaternion

$$q = \left(\cos \frac{\theta}{2}, \sin \frac{\theta}{2} \hat{n} \right) \quad (43)$$

6.7 Double Cover of $SO(3)$

Unit quaternions live on S^3 (the 3-sphere in $\mathbb{R}^4$). The map $q \mapsto R(q)$ from unit quaternions to the rotation group $SO(3)$ is a **2-to-1 surjection**: both q and $-q$ produce the same rotation matrix.

$$S^3 / \{\pm 1\} \cong SO(3) \quad (44)$$

This is why the half-angle appears: a full 2π physical rotation corresponds to only a π traversal on S^3 . A 4π rotation is needed to return to the *same* quaternion.

Practical consequence

Because q and $-q$ are the same rotation, algorithms that interpolate or average quaternions must check the sign of $q_1 \cdot q_2$ and negate one quaternion if needed to ensure the shortest-path interpretation.

6.8 Quaternion Derivative — Rotational Kinematics

If a rigid body has angular velocity $\vec{\omega}$ expressed in the body frame, the quaternion evolves as:

$$\dot{q} = \frac{1}{2} q \otimes \omega_q, \quad \omega_q = (0, \vec{\omega}) \quad (45)$$

In matrix form:

$$\dot{q} = \frac{1}{2} \Omega q, \quad \Omega = \begin{pmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{pmatrix} \quad (46)$$

This ODE is integrated in INS/AHRS systems to propagate attitude.

6.9 SLERP — Spherical Linear Interpolation

To smoothly interpolate between orientations q_0 and q_1 :

$$\text{slerp}(q_0, q_1, t) = q_0 \frac{\sin((1-t)\Omega)}{\sin \Omega} + q_1 \frac{\sin(t\Omega)}{\sin \Omega}, \quad t \in [0, 1] \quad (47)$$

where $\cos \Omega = q_0 \cdot q_1$ (4-D dot product). SLERP traverses the great circle on S^3 , producing constant angular velocity—unlike linear interpolation of Euler angles.

6.10 Quaternion Error and Attitude Control

The rotation from q_a to q_b is:

$$q_{\text{err}} = q_b \otimes q_a^* \quad (48)$$

For a proportional attitude controller the error signal is commonly extracted as:

$$\vec{e} = 2 \operatorname{sign}(w_e) \vec{v}_e, \quad (w_e, \vec{v}_e) = q_{\text{err}} \quad (49)$$

For small angles this approximates the rotation-vector error.

6.11 Exponential and Logarithmic Maps

These connect unit quaternions to the Lie algebra $\mathfrak{so}(3)$.

Exponential map (rotation vector $\rightarrow$ quaternion):

$$\exp(\vec{\theta}) = \left(\cos \frac{\|\vec{\theta}\|}{2}, \frac{\vec{\theta}}{\|\vec{\theta}\|} \sin \frac{\|\vec{\theta}\|}{2} \right) \quad (50)$$

Logarithmic map (quaternion $\rightarrow$ rotation vector):

$$\log(q) = 2 \operatorname{atan2}(\|\vec{v}\|, w) \frac{\vec{v}}{\|\vec{v}\|} \quad (51)$$

These maps are essential for:

- **Optimisation on $SO(3)$:** perturb in the tangent space, then map back.
- **Kalman filtering:** attitude errors live in three-dimensional tangent space (multiplicative EKF / MEKF).
- **Averaging rotations:** compute the mean in log space.

6.12 Euler Angles vs. Quaternions

6.13 Practical Tips for GNC

1. **Normalisation.** After every integration step, re-normalise: $q \leftarrow q/\|q\|$.
2. **Convention.** Hamilton (w first) vs. JPL (w last) changes composition order. Know which your codebase uses.
3. **Sign convention.** Enforce $w > 0$ to avoid discontinuities in logging and control.

Property	Euler Angles	Quaternions
Parameters	3	4 (with 1 constraint)
Singularities	Gimbal lock at $\pm 90^\circ$ pitch	None
Composition	Trigonometry-heavy	Single multiplication
Interpolation	Non-uniform, jerky	SLERP (smooth)
Numerical drift	N/A	Re-normalise: $q \leftarrow q/\ q\ $
Human intuition	Easy	Harder initially

Table 13: Comparison of Euler angles and quaternions.

4. **Small-angle approximation.** For $\theta \ll 1$: $q \approx (1, \vec{\theta}/2)$.
5. **MEKF.** Represent attitude error as a 3-vector (rotation vector) to avoid the rank deficiency of estimating 4 quaternion components with a 4×4 covariance.

6.14 MATLAB Examples

```

1 % Define a unit quaternion: 45-deg rotation about Z
2 theta = deg2rad(45);
3 q = [cos(theta/2), 0, 0, sin(theta/2)]; % [w x y z]
4
5 % Rotate a vector using quaternion sandwich product
6 v = [1; 0; 0];
7 v_q = [0, v']; % pure quaternion
8 v_rot = quatmultiply(quatmultiply(q, v_q), quatconj(q));
9 v_rotated = v_rot(2:4) % extract vector part

```

Listing 3: Quaternion rotation of a vector.

```

1 % quaternion -> Euler ZYX
2 eul = quat2eul(q, 'ZYX'); % [yaw pitch roll]
3
4 % Euler ZYX -> quaternion
5 q_back = eul2quat(eul, 'ZYX');
6 assert(norm(q - q_back) < 1e-12);

```

Listing 4: Quaternion to Euler (ZYX) and back.

```

1 q0 = [1 0 0 0]; % identity
2 q1 = [cos(pi/4) 0 sin(pi/4) 0]; % 90 deg about Y
3 t = 0.5;
4 q_mid = quatinterp(q0, q1, t, 'slerp');

```

Listing 5: SLERP between two orientations.

7 Denavit–Hartenberg (DH) Parameters

The Denavit–Hartenberg convention provides a systematic way to describe the geometry of a serial kinematic chain (robot arm, excavator boom, crane, etc.) using only **four parameters per joint**. Combined with a simple 4×4 homogeneous transformation, DH parameters let you compute the position and orientation of the end effector from the joint angles alone.

7.1 The Four DH Parameters

Each link i in the chain is described by exactly four quantities:

Symbol	Name	Definition
θ_i	Joint angle	Angle about Z_{i-1} from X_{i-1} to X_i (measured in the X_{i-1} – Y_{i-1} plane).
d_i	Link offset	Distance along Z_{i-1} from the origin of frame $i-1$ to the intersection of Z_{i-1} and X_i .
a_i	Link length	Distance along X_i from Z_{i-1} to Z_i (the “common normal” length).
α_i	Link twist	Angle about X_i from Z_{i-1} to Z_i (measured in the Y_i – Z_i plane).

Table 14: The four Denavit–Hartenberg parameters for link i .

DH frame assignment rules

1. The Z_{i-1} axis lies along the axis of joint i .
2. The X_i axis is the common normal from Z_{i-1} to Z_i , pointing away from Z_{i-1} .
3. The Y_i axis completes the right-hand frame: $Y_i = Z_i \times X_i$.
4. The origin of frame i is at the intersection of X_i and Z_i (or at the foot of the common normal on Z_i).

7.2 The DH Transformation Matrix

The homogeneous transformation from frame $i-1$ to frame i is:

$${}^{i-1}T_i = \underbrace{\text{Rot}_Z(\theta_i) \text{Trans}_Z(d_i)}_{\text{along } Z_{i-1}} \underbrace{\text{Trans}_X(a_i) \text{Rot}_X(\alpha_i)}_{\text{along } X_i} \quad (52)$$

Multiplying these four elementary transforms out:

$${}^{i-1}T_i = \begin{pmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (53)$$

The upper-left 3×3 block is the rotation ${}^{i-1}R_i$; the right-most column (top three entries) is the translation ${}^{i-1}\vec{p}_i$.

Reading the matrix

- **Row 3, col 4:** d_i — the offset along the previous joint axis.
- **Rows 1–2, col 4:** $a_i \cos \theta_i$ and $a_i \sin \theta_i$ — the link-length contribution projected by the joint angle.
- **Column 3:** contains $\sin \alpha_i$, $\cos \alpha_i$ — the twist between successive Z axes.

7.3 Rotary (Revolute) vs. Linear (Prismatic) Joints

The four DH parameters have different roles depending on the joint type:

	Revolute (Rotary) Joint	Prismatic (Linear) Joint
Joint variable	θ_i (angle)	d_i (displacement)
Fixed parameters	d_i, a_i, α_i	θ_i, a_i, α_i
Motion	Rotation about Z_{i-1}	Translation along Z_{i-1}
DH matrix changes	$\cos \theta_i, \sin \theta_i$ vary	d_i varies (row 3, col 4)
Typical use	Robot elbow, excavator boom pivot, steering joint	Hydraulic cylinder, telescoping mast, linear actuator

Table 15: DH parameter roles for revolute vs. prismatic joints.

Identifying the joint variable

For **any** joint, exactly **one** of the four DH parameters is the joint variable; the other three are fixed by the mechanism geometry.

- Revolute: the joint variable is θ_i .
- Prismatic: the joint variable is d_i .

All higher-order joints (helical, cylindrical, spherical) can be decomposed into combinations of these two.

Revolute Joint — Details

For a revolute joint at link i :

- θ_i is the **unknown** (measured by encoder or commanded by controller).
- d_i is a fixed offset (often zero for planar mechanisms).
- a_i is the physical link length (boom segment, forearm, etc.).
- α_i is the fixed twist angle between consecutive axes (e.g. 0° for parallel axes, 90° for perpendicular).

Prismatic Joint — Details

For a prismatic joint at link i :

- d_i is the **unknown** (cylinder extension).
- θ_i is a fixed constant (often 0°).
- a_i is typically 0 (the slide direction *is* the Z_{i-1} axis).
- α_i sets the twist to the next joint's axis.

7.4 Forward Kinematics — End Effector Position

For an n -joint serial chain the end effector pose (position + orientation) in the base frame is:

$${}^0T_n = {}^0T_1 {}^1T_2 {}^2T_3 \cdots {}^{n-1}T_n = \prod_{i=1}^n {}^{i-1}T_i \quad (54)$$

Each ${}^{i-1}T_i$ is the 4×4 DH matrix from Equation (53), evaluated with the current joint variable (θ_i or d_i).

From the resulting 4×4 matrix:

$${}^0T_n = \begin{pmatrix} {}^0R_n & {}^0\vec{p}_n \\ \vec{0}^T & 1 \end{pmatrix} \quad (55)$$

- ${}^0\vec{p}_n = \begin{pmatrix} p_x \\ p_y \\ p_z \end{pmatrix}$ is the **end effector position** in the base frame.
- 0R_n is the **end effector orientation** (a 3×3 rotation matrix that can be converted to Euler angles or a quaternion using the formulas in earlier sections).

Worked Example: 2-Link Planar Arm (Two Revolute Joints)

Link	θ_i	d_i	a_i	α_i
1	θ_1 (variable)	0	L_1	0°
2	θ_2 (variable)	0	L_2	0°

Table 16: DH table for a 2-DOF planar arm.

With $\alpha_i = 0$ and $d_i = 0$, each DH matrix simplifies to:

$${}^{i-1}T_i = \begin{pmatrix} \cos \theta_i & -\sin \theta_i & 0 & L_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i & 0 & L_i \sin \theta_i \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (56)$$

The end effector position is:

$$\begin{pmatrix} p_x \\ p_y \end{pmatrix} = \begin{pmatrix} L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{pmatrix} \quad (57)$$

This is the classic 2-link forward kinematics result.

Worked Example: Revolute + Prismatic (RP)

Consider a rotary base followed by a telescoping arm:

Link	θ_i	d_i	a_i	α_i
1 (revolute)	θ_1 (variable)	0	0	90°
2 (prismatic)	0°	d_2 (variable)	0	0°

Table 17: DH table for a revolute–prismatic (RP) mechanism.

Joint 1 rotates about the base Z_0 ; joint 2 extends along Z_1 (perpendicular to Z_0 due to $\alpha_1 = 90^\circ$).

$${}^0T_1 = \begin{pmatrix} \cos \theta_1 & 0 & \sin \theta_1 & 0 \\ \sin \theta_1 & 0 & -\cos \theta_1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad {}^1T_2 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_2 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (58)$$

End effector position:

$${}^0\vec{p}_2 = \begin{pmatrix} d_2 \sin \theta_1 \\ -d_2 \cos \theta_1 \\ 0 \end{pmatrix} \quad (59)$$

The prismatic extension d_2 is projected through the base rotation θ_1 —exactly how a turret-mounted telescoping arm behaves.

7.5 Summary of DH Procedure

1. **Number the joints** 1 to n from base to end effector.
2. **Assign Z axes:** Z_{i-1} along the axis of joint i .
3. **Assign X axes:** X_i along the common normal from Z_{i-1} to Z_i .
4. **Fill the DH table:** for each link, record θ_i , d_i , a_i , α_i .
5. **Mark the joint variable:** θ_i for revolute, d_i for prismatic.
6. **Build ${}^{i-1}T_i$:** plug each row of the table into Equation (53).
7. **Multiply:** ${}^0T_n = \prod {}^{i-1}T_i$ to get the end effector pose.

DH vs. Euler angles

DH parameters describe the *geometry between successive joints* (link lengths, offsets, twists), while Euler angles describe the *orientation of a single body*. A serial chain's end effector orientation (extracted from 0R_n) can be converted to Euler angles or quaternions using the formulas in Sections 2 and 6.

```

1 function T = DH(theta, d, a, alpha)
2 % Build 4x4 DH transformation matrix
3   ct = cos(theta); st = sin(theta);
4   ca = cos(alpha); sa = sin(alpha);
5   T = [ct, -st*ca, st*sa, a*ct;
6         st,  ct*ca, -ct*sa, a*st;
7         0,   sa,   ca,   d;
8         0,   0,   0,   1];
9 end
10
11 % Example: 3-link planar arm (all revolute, alpha=0, d=0)
12 L = [0.5, 0.4, 0.3]; % link lengths [m]
13 q = deg2rad([30, 45, -20]); % joint angles [rad]
14
15 T_total = eye(4);
16 for i = 1:3
17     T_total = T_total * DH(q(i), 0, L(i), 0);
18 end
19
20 p_ee = T_total(1:3, 4); % end effector position
21 R_ee = T_total(1:3, 1:3); % end effector orientation
22 eul = rotm2eul(R_ee, 'ZYX'); % convert to Euler angles

```

Listing 6: Forward kinematics of a 3-DOF arm using DH parameters in MATLAB.